UQAR

TECHNOLOGIES DE L'INFOROUTE

INF37407 (MS)

UNIVERSITÉ DU QUÉBEC À RIMOUSKI - CAMPUS LÉVIS

DÉPARTEMENT DE MATHÉMATIQUES, D'INFORMATIQUE ET DE GÉNIE

RAPPORT

Travail Pratique 1 : TravelHub

ÉQUIPE :
Nom & Prénom
Nom & Prénom
Nom & Prénom

PROFESSEUR :
Yacine Yaddaden, Ph. D.

Date : 24 octobre 2026

Table des matières

1 Introduction **5**
1.1 Contexte 5
1.2 Objectifs 5
1.3 Périmètre 5
1.4 Organisation du rapport 5

2 Environnement et technologies **6**
2.1 Versions retenues 6
2.2 Justification de Django 5.2 LTS 6
2.3 Configuration 7

3 Architecture **7**
3.1 Applications Django 7
3.2 Arborescence 8
3.3 Répartition des responsabilités 8
3.4 Flux d'une requête 8

4 Modèle de données **8**
4.1 Diagramme entité-association 8
4.2 Relations 11
4.3 Utilisateur personnalisé 11
4.4 Contraintes d'intégrité 12
4.5 Migrations 13

5 Gestion des voyages, destinations, activités et participants **13**
5.1 QuerySets réutilisables 13
5.2 Relations inverses et préchargement 14
5.3 Agrégations 15
5.4 Services métier 15
5.5 Administration et données de démonstration 16

6 Intégrations externes **17**
6.1 Open-Meteo (REST) 17
6.2 Countries (GraphQL) 17
6.3 Couche HTTP commune 18
6.4 Cache 18
6.5 Agrégation et dégradation partielle 19

7 API REST **20**
7.1 Points d'accès 20
7.2 Serializers 21

7.3 ViewSets et routage 21
7.4 Filtres, recherche, tri et pagination 22
7.5 Codes HTTP et format des erreurs 23
7.6 Documentation OpenAPI 23

8 API GraphQL **24**
8.1 Schéma 24
8.2 Réutilisation de la logique REST 25
8.3 Authentification, limites et erreurs 25
8.4 Durcissement et performance 26
8.5 Exemple 26
8.6 Comparaison REST et GraphQL 27

9 Validation, transactions et intégrité **28**
9.1 Trois niveaux de validation 28
9.2 Appliquer `Model.clean()` depuis DRF 29
9.3 Contraintes violées malgré la validation 30
9.4 Transactions 30
9.5 Démonstration du rollback 30

10 Sécurité **31**
10.1 Authentification par JWT 31
10.2 Permissions par rôle 32
10.3 Contrôle d'accès au niveau objet 32
10.4 Configuration de production 32
10.5 OWASP API Security Top 10 33
10.6 Risques résiduels 34

11 Tests **34**
11.1 Stratégie 34
11.2 Couverture fonctionnelle 35
11.3 Résultats et couverture 35
11.4 Collection Postman 36
11.5 Qualité du code 37

12 Industrialisation **37**
12.1 Docker 37
12.2 Intégration continue 37
12.3 Journaux et contrôle de santé 38
12.4 Sauvegarde et restauration 38

13 Conclusion **39**
13.1 Bilan 39
13.2 Limites 39
13.3 Pistes pour le TP2 39

1 Introduction

1.1 Contexte

TravelHub est une plateforme de planification de voyages collaboratifs. Un utilisateur crée un voyage, le découpe en destinations, puis planifie des activités à chaque étape. Il peut partager le voyage avec d'autres utilisateurs, à qui il attribue un rôle : propriétaire, éditeur ou lecteur. Chaque destination est enrichie de données externes : la météo du séjour et une fiche descriptive du pays.
Ce rapport présente le back-end réalisé pour le premier travail pratique. Il décrit les choix techniques, l'architecture, le modèle de données, les deux API exposées, l'intégration des services externes, la sécurité et la stratégie de test. Chaque affirmation renvoie au code du dépôt.

1.2 Objectifs

Le travail poursuit six objectifs :

- modéliser les données dans PostgreSQL, avec des relations 1-N et N-N et des contraintes d'intégrité vérifiées par la base elle-même ;
- exposer les mêmes fonctions par une API REST (Django REST Framework) et par une API GraphQL (Graphene-Django) ;
- consommer deux API publiques : Open-Meteo, de style REST, et Countries, de style GraphQL ;
- protéger l'accès par jetons JWT et par des permissions fondées sur le rôle, en suivant l'OWASP API Security Top 10 [13] ;
- garantir la cohérence des données par la validation à plusieurs niveaux et par les transactions ;
- vérifier le tout par une suite de tests automatisés et par une collection Postman.

1.3 Périmètre

Le TP1 porte sur le back-end seul. L'interface utilisateur relève du TP2. L'API constitue donc l'unique interface du système. Elle est exercée par Swagger UI, GraphiQL, Postman et l'administration de Django. Le dépôt contient aussi une part d'industrialisation : image Docker, intégration continue, journaux structurés et sauvegardes. Cette part n'est pas exigée par l'énoncé ; elle prépare le déploiement du TP2.

1.4 Organisation du rapport

La section 2 justifie les versions retenues. Les sections 3 et 4 présentent l'architecture et le modèle de données. La section 5 décrit la logique métier. La section 6 traite des services externes. Les sections 7 et 8 présentent les deux API et les comparent. Les

sections 9 à 11 couvrent l'intégrité, la sécurité et les tests. La section 12 décrit le déploiement, et la section 13 conclut.

2 Environnement et technologies

2.1 Versions retenues

Le tableau 1 reprend les versions figées dans `requirements.txt` et `requirements-dev.txt`. Chaque dépendance est épinglée à une version exacte : deux installations produisent le même environnement.

TABLE 1 – Versions des outils et des bibliothèques.

Composant	Version	Rôle
Python	3.14	Langage d'exécution
PostgreSQL	18	Base de données relationnelle [14]
Django	5.2.18 (LTS)	Cadriciel web, ORM, administration [4]
Django REST Framework	3.18.3	API REST [2]
SimpleJWT	5.5.1	Jetons JWT, liste noire des jetons révoqués [9]
drf-spectacular	0.30.0	Schéma OpenAPI 3, Swagger UI, ReDoc [5]
Graphene-Django	3.2.3	API GraphQL [7]
django-filter	26.2	Filtres des listes
django-cors-headers	4.9.0	Politique CORS
django-environ	0.14.0	Configuration par variables d'environnement
psycopg	3.3.6	Pilote PostgreSQL
requests	2.34.2	Client HTTP des services externes
Gunicorn, WhiteNoise	26.2.0, 6.12.0	Serveur WSGI et fichiers statiques (production)
pytest, pytest-django	9.1.1, 4.14.0	Tests
pytest-cov	7.1.0	Couverture de code
responses	0.26.3	Simulation des appels HTTP externes
Ruff	0.17.1	Analyse statique et formatage

Le poste de développement fonctionne sous Windows 11. Les scripts d'exploitation sont écrits en PowerShell. La base est administrée avec pgAdmin 4.

2.2 Justification de Django 5.2 LTS

Le critère de choix était la dernière version stable *et compatible* avec l'ensemble des dépendances. Django 6.1 existait au moment du choix. Cependant, Graphene-Django 3.2.3 et SimpleJWT 5.5.1 ne déclarent pas encore la prise en charge de Django 6. Django 5.2 est une version à support étendu (LTS), maintenue jusqu'en avril 2028, et prend

officiellement en charge Python 3.14. Ce choix privilégie la stabilité de l'ensemble sur la nouveauté du cadriciel.

2.3 Configuration

Les paramètres sont découpés en quatre modules (tableau 2). Les valeurs sensibles ne figurent jamais dans le code : `django-environ` les lit dans l'environnement, puis dans un fichier `.env` ignoré par Git. Le fichier `.env.example` documente chaque variable.

TABLE 2 – Modules de paramètres.

Module	Usage
`config.settings.base`	Paramètres communs : applications, DRF, JWT, GraphQL, journaux
`config.settings.dev`	Développement : `DEBUG`, GraphiQL et introspection activés
`config.settings.test`	Tests : hachage rapide des mots de passe, cache en mémoire, journaux sur la console
`config.settings.prod`	Production : HTTPS, HSTS, cookies sécurisés, JSON seulement, WhiteNoise

3 Architecture

3.1 Applications Django

Le projet suit la structure recommandée par Django : un projet de configuration (`config`) et des applications à responsabilité unique, regroupées sous `apps/` (tableau 3).

TABLE 3 – Applications du projet.

Application	Responsabilité
`config`	Paramètres, routage principal, schéma GraphQL racine, points d'entrée WSGI et ASGI
`apps.core`	Éléments transversaux : `TimeStampedModel`, permissions par rôle, gestionnaire d'exceptions, vue GraphQL, journaux, contrôle de santé
`apps.accounts`	Utilisateur personnalisé, inscription, jetons JWT, profil
`apps.trips`	Voyages et participants, services métier (création, clonage, statistiques)
`apps.itinerary`	Destinations et activités, coordonnées géographiques
`apps.integrations`	Clients Open-Meteo et Countries, couche HTTP commune, agrégateur

Chaque application métier contient les mêmes fichiers : `models.py`, `serializers.py`, `views.py`, `urls.py`, `filters.py`, `schema.py` (GraphQL), `admin.py`, `services.py` et un dossier `tests/`. Cette régularité facilite la navigation dans le code.

3.2 Arborescence

Extrait 1 – Arborescence du dépôt (simplifiée).

```
TravelHub/
|-- config/              settings/ (base, dev, test, prod), urls.py, schema.py
|-- apps/
|   |-- core/            permissions, exceptions, vue GraphQL, journaux
|   |-- accounts/        User, inscription, JWT
|   |-- trips/           Trip, TripParticipant, services
|   |-- itinerary/       Destination, Activity
|   `-- integrations/    services/ (http, open_meteo, countries, aggregator)
|-- postman/             collection v2.1 et environnement
|-- scripts/             backup.ps1, restore.ps1, db/
|-- .github/workflows/   ci.yml
|-- Dockerfile  docker-compose.yml  gunicorn.conf.py
`-- requirements.txt  requirements-dev.txt  pytest.ini  .env.example
```

3.3 Répartition des responsabilités

Le code respecte quatre principes.

1. **Des vues minces.** Une vue authentifie, choisit le queryset, applique la permission, puis délègue. Elle ne contient pas de règle métier.
2. **Des services explicites.** Les opérations qui touchent plusieurs tables (`create_trip`, `clone_trip`, `add_participant`) vivent dans `services.py`. Les API REST et GraphQL les appellent toutes deux.
3. **Des règles portées par le modèle et par la base.** Les règles de dates sont écrites dans `Model.clean()`. Les invariants sont doublés par des contraintes PostgreSQL.
4. **Des erreurs centralisées.** Un gestionnaire unique (`apps/core/exceptions.py`) produit la même enveloppe JSON pour toutes les erreurs.

3.4 Flux d'une requête

La figure 1 suit une requête `PATCH /api/trips/12/` de bout en bout.
L'ordre de ces étapes porte la sécurité. Le filtrage par `visible_to()` précède la permission objet. Un voyage d'autrui n'existe donc pas pour l'utilisateur : il reçoit 404, et non 403, ce qui ne révèle pas l'existence de la ressource.

4 Modèle de données

4.1 Diagramme entité-association

Le modèle compte cinq tables métier (figure 2). Les tables techniques de Django et de SimpleJWT (sessions, permissions, liste noire des jetons) n'y figurent pas. Les champs

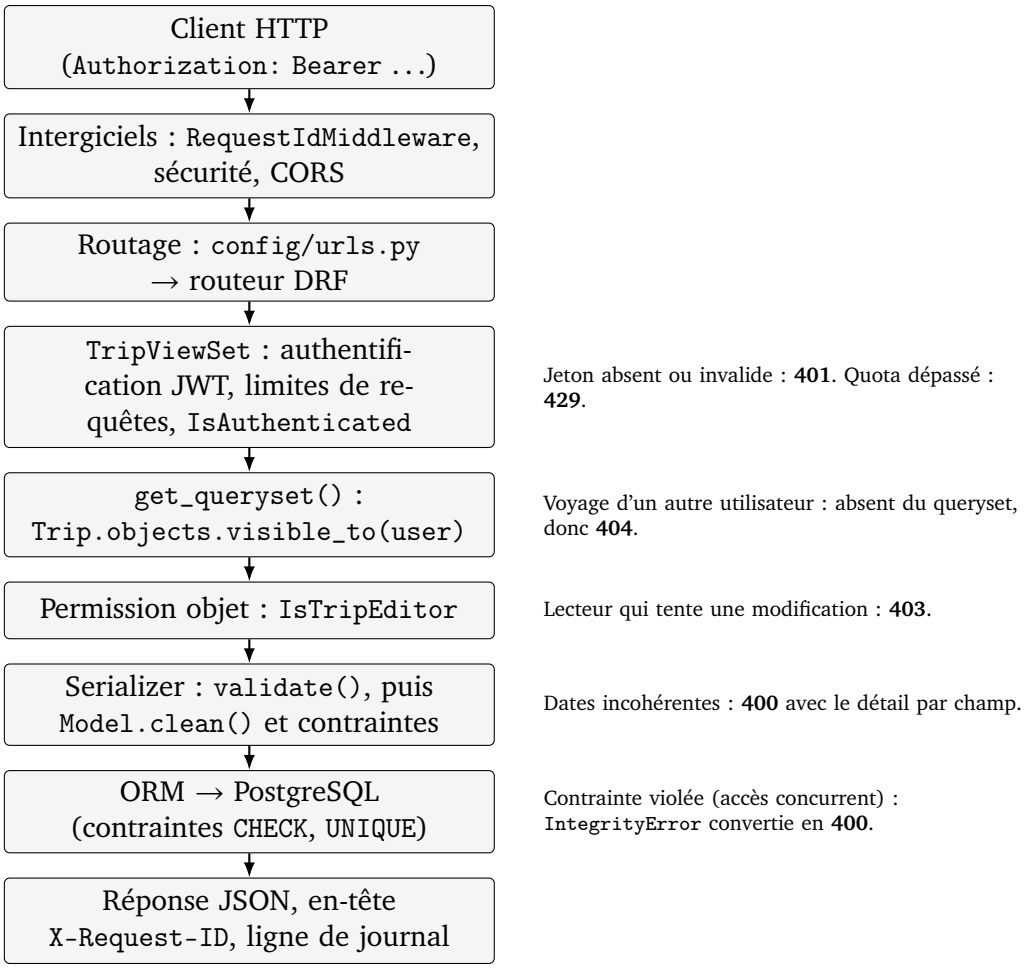


FIGURE 1 – Traitement d'une modification de voyage. Toute exception est convertie par `exception_handler` en enveloppe `{"error": ...}`.

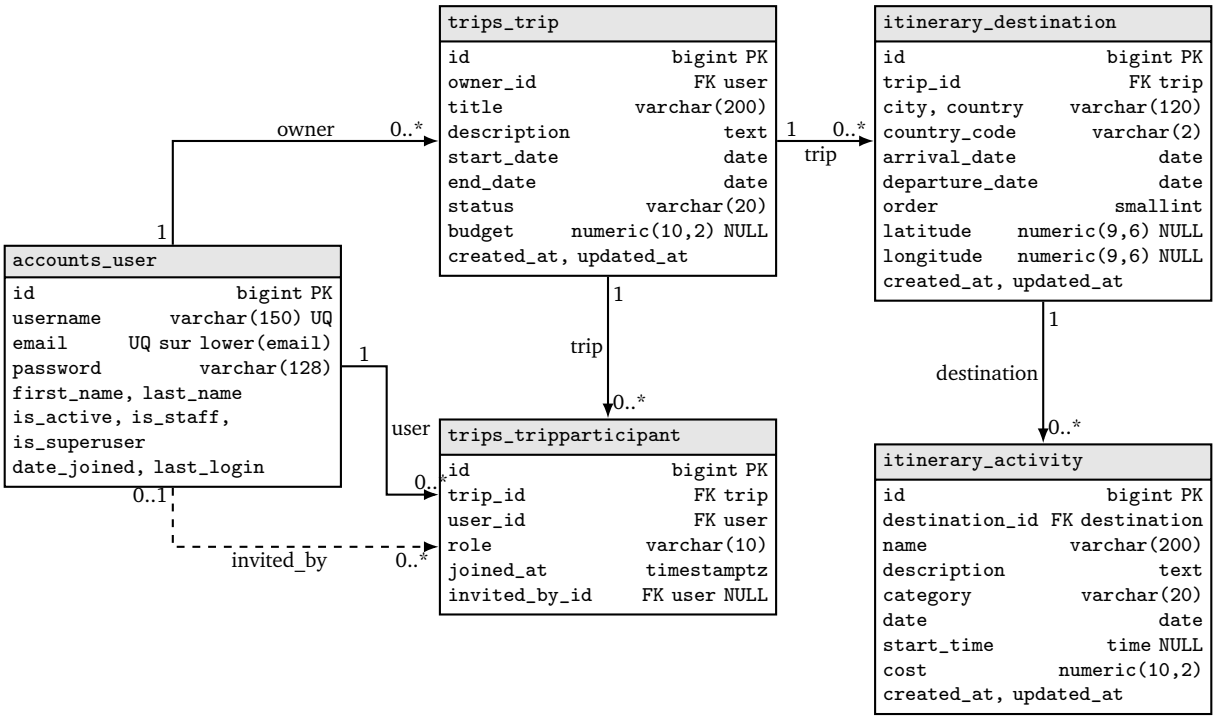


FIGURE 2 – Modèle de données de TravelHub. Les flèches vont de la table référencée vers la table qui porte la clé étrangère. La relation N-N entre utilisateurs et voyages passe par `trips_tripparticipant`.

`created_at` et `updated_at` proviennent du modèle abstrait `TimeStampedModel` de `apps.core`.

La figure 3 montre le même schéma, produit par l'outil ERD de pgAdmin à partir de la base réelle.

Capture à insérer :
`figures/erd-pgadmin.png`

FIGURE 3 – Diagramme ERD généré par pgAdmin 4 (*Tools → ERD Tool*) sur la base `travelhub`.

4.2 Relations

Le tableau 4 détaille chaque relation. Le nom inverse (`related_name`) est choisi pour se lire naturellement : `user.owned_trips`, `trip.destinations`, `destination.activities`.

TABLE 4 – Relations du modèle.

Relation	Type	Suppression	Accès inverse
`Trip.owner` → `User`	1-N	`CASCADE`	`user.owned_trips`
`Trip.participants` ↔ `User`	N-N	—	`user.trips`, via `TripParticipant`
`TripParticipant.trip` → `Trip`	1-N	`CASCADE`	`trip.memberships`
`TripParticipant.user` → `User`	1-N	`CASCADE`	`user.trip_memberships`
`TripParticipant.invited_by` → `User`	1-N	`SET_NULL`	`user.sent_trip_invitations`
`Destination.trip` → `Trip`	1-N	`CASCADE`	`trip.destinations`
`Activity.destination` → `Destination`	1-N	`CASCADE`	`destination.activities`

La relation N-N mérite une précision. Une simple table de jointure ne suffit pas : l'association porte des attributs (rôle, date d'ajout, auteur de l'invitation). Elle est donc modélisée par un modèle intermédiaire explicite, déclaré avec `through`. Comme `TripParticipant` possède deux clés étrangères vers `User`, l'option `through_fields` désigne celles qui relient le voyage à ses participants.

Extrait 2 – Relation N-N avec modèle intermédiaire (`apps/trips/models.py`, extrait).

```
participants = models.ManyToManyField(
    settings.AUTH_USER_MODEL,
    through="TripParticipant",
    # TripParticipant a deux FK vers User (user, invited_by) : on indique lesquelles
    # relient le voyage à ses participants.
    through_fields=("trip", "user"),
    related_name="trips",
)
```

La suppression en cascade suit la composition : un voyage possède ses destinations, qui possèdent leurs activités. En revanche, la suppression de l'auteur d'une invitation ne doit pas retirer le participant invité ; `invited_by` passe alors à `NULL`.

4.3 Utilisateur personnalisé

Le modèle `accounts.User` hérite d'`AbstractUser`. Il a été déclaré dans `AUTH_USER_MODEL` avant la première migration, comme le recommande Django : changer de modèle utilisateur après coup est très coûteux. Le courriel est obligatoire et unique sans tenir compte de la casse, grâce à une contrainte d'unicité sur l'expression `Lower("email")`. Le

courriel est aussi normalisé en minuscules dans `save()`, car `createsuperuser` n'appelle pas `clean()`.

4.4 Contraintes d'intégrité

Les règles essentielles sont vérifiées par PostgreSQL lui-même (tableau 5). La base reste ainsi cohérente même si une écriture contourne la validation applicative, par exemple lors de deux requêtes simultanées. Chaque contrainte porte un nom et un message d'erreur en français, réutilisé par le gestionnaire d'exceptions (section 9).

TABLE 5 – Contraintes déclarées dans les modèles.

Nom	Type	Règle
`accounts_user_email_ci_unique`	`UNIQUE`	`lower(email)` unique
`trips_trip_end_after_start`	`CHECK`	`end_date >= start_date`
`trips_trip_budget_non_negative`	`CHECK`	budget nul ou positif
`trips_trip_status_valid`	`CHECK`	statut parmi les cinq valeurs
`trips_participant_unique_trip_user`	`UNIQUE`	un utilisateur au plus une fois par voyage
`trips_participant_single_owner`	`UNIQUE` partiel	un seul `owner` par voyage (`WHERE role = 'owner'`)
`trips_participant_role_valid`	`CHECK`	rôle parmi `owner`, `editor`, `viewer`
`itinerary_destination_unique_trip_order`	`UNIQUE`	une position par destination dans un voyage
`itinerary_destination_departure_after_arrival`	`CHECK`	`departure_date >= arrival_date`
`itinerary_destination_country_code_iso2`	`CHECK`	code pays conforme à `^[A-Z]{2}$`
`itinerary_destination_coordinates_valid`	`CHECK`	latitude et longitude fournies ensemble, dans leurs bornes
`itinerary_activity_cost_non_negative`	`CHECK`	`cost >= 0`
`itinerary_activity_category_valid`	`CHECK`	catégorie parmi les huit valeurs

Deux points de conception méritent d'être relevés. D'abord, la contrainte `trips_participant_single_owner` est un index unique *partiel* : il ne porte que sur les lignes où `role = 'owner'`. Ensuite, la contrainte sur les coordonnées teste explicitement

`IS NOT NULL`. En SQL, une condition `CHECK` évaluée à `NULL` est considérée comme respectée ; sans ce test, une latitude seule passerait.

Extrait 3 – Contraintes du voyage (`apps/trips/models.py`, extrait).

```
class Meta:
    ordering = ["start_date", "pk"]
    indexes = [models.Index(fields=["start_date"], name="trips_trip_start_date_idx")]
    constraints = [
        models.CheckConstraint(
            condition=Q(end_date__gte=F("start_date")),
            name="trips_trip_end_after_start",
            violation_error_message=(
                "La date de fin doit être postérieure ou égale à la date de début."
            ),
        ),
        models.CheckConstraint(
            condition=Q(budget__isnull=True) | Q(budget__gte=0),
            name="trips_trip_budget_non_negative",
            violation_error_message="Le budget ne peut pas être négatif.",
        ),
        # ... trips_trip_status_valid
    ]
```

Les montants utilisent `DecimalField` (`numeric` en SQL) et jamais `float` : un coût de 0,10 $ doit rester exact. Les choix (statut, rôle, catégorie) sont des `TextChoices` ; ils sont doublés d'une contrainte `CHECK`, car Django ne vérifie les choix qu'à la validation, jamais en base.

4.5 Migrations

Chaque application possède une migration initiale (`0001_initial.py`), générée par Django 5.2.18 avec `makemigrations`. La commande `sqlmigrate` permet de relire le SQL produit, notamment les contraintes et l'index partiel. La commande `migrate -check` sert à vérifier qu'une base restaurée est à jour (section 12). Dans Docker, le service `web` applique les migrations à chaque démarrage, avant de lancer Gunicorn.

5 Gestion des voyages, destinations, activités et participants

5.1 QuerySets réutilisables

Les requêtes fréquentes sont regroupées dans un `QuerySet` personnalisé, exposé par `Trip.objects` (`TripQuerySet.as_manager()`). Elles se combinent par chaînage.

TABLE 6 – Méthodes de `TripQuerySet`.

Méthode	Résultat
`visible_to(user)`	Voyages dont l'utilisateur est propriétaire ou participant, quel que soit son rôle ; vide pour un anonyme
`editable_by(user)`	Voyages que l'utilisateur peut modifier : propriétaire, rôle `owner` ou `editor`
`upcoming()`	Voyages à venir, hors voyages annulés ou terminés, triés par date
`with_totals()`	Annotations `destination_count`, `activity_count` et `total_cost`

La méthode `visible_to()` est le point de départ de toute vue et de tout resolver GraphQL. Elle utilise une sous-requête `Exists()` plutôt qu'une jointure sur les participants. Une jointure dupliquerait chaque voyage autant de fois qu'il a de participants, et fausserait les sommes de `with_totals()`.

Extrait 4 – Voyages visibles et totaux (`apps/trips/models.py`, extrait).

```python
def visible_to(self, user):
    if user is None or not user.is_authenticated:
        return self.none()
    # Exists() plutôt qu'une jointure : aucun doublon, et with_totals() reste exact.
    membership = TripParticipant.objects.filter(trip=OuterRef("pk"), user=user)
    return self.filter(Q(owner=user) | Exists(membership))

def with_totals(self):
    return self.annotate(
        destination_count=Count("destinations", distinct=True),
        activity_count=Count("destinations__activities", distinct=True),
        total_cost=Coalesce(
            Sum("destinations__activities__cost"),
            Value(0),
            output_field=DecimalField(max_digits=12, decimal_places=2),
        ),
    )
```

Le calcul de `total_cost` suit la chaîne voyage → destinations → activités. Chaque activité n'apparaît qu'une fois dans cette jointure : la somme est exacte. Le code interdit d'y ajouter un `Count()` sur une seconde relation, car cette jointure supplémentaire multiplierait les lignes. Pour la même raison, le tri de la liste des voyages ne propose aucun champ de relation.

5.2 Relations inverses et préchargement

Les relations inverses du tableau 4 évitent d'écrire des requêtes explicites : `trip.memberships.select_related("user")` liste les participants, `destination.activities.all()` les activités d'une étape. Le filtrage à travers les relations reste possible dans les deux sens, par exemple `Activity.objects.filter(destination__trip=trip)`.

Le préchargement empêche le problème des « N+1 requêtes ». La liste des voyages

charge le propriétaire par `select_related("owner")`. Le détail d'un voyage précharge ses destinations, triées par position, puis leurs activités, à l'aide d'un objet `Prefetch`. Le nombre de requêtes SQL ne dépend donc pas du nombre de destinations.

5.3 Agrégations

L'action `GET /api/trips/{id}/stats/` calcule le budget restant et la répartition des coûts par catégorie. Tous les calculs sont confiés à PostgreSQL : `aggregate()` pour les totaux, `values().annotate()` pour le regroupement.

Extrait 5 – Statistiques par catégorie (`apps/trips/services.py`, extrait).

```
activities = Activity.objects.filter(destination__trip=trip)
totals = activities.aggregate(count=Count("pk"), total=Sum("cost"), average=Avg("cost"))
by_category = (
    activities.values("category")
    .annotate(count=Count("pk"), total_cost=Sum("cost"), average_cost=Avg("cost"))
    # order_by() explicite : l'ordre par défaut d'Activity ajouterait date au GROUP BY.
    .order_by("-total_cost", "category")
)
```

Le commentaire de l'extrait 5 signale un piège de l'ORM. Le tri par défaut d'`Activity` porte sur la date. S'il était conservé, Django ajouterait la date au `GROUP BY`, et chaque catégorie serait éclatée par jour.

5.4 Services métier

Création. Le service `create_trip()` valide le voyage (`full_clean()`), l'enregistre, puis inscrit le propriétaire comme participant `owner`, dans une seule transaction. Un signal `post_save` aurait produit le même effet, mais de façon implicite. Le service rend l'effet de bord visible à l'appel.

Clonage. Le service `clone_trip()` copie un voyage, ses destinations et leurs activités dans un nouveau brouillon. Il utilise `bulk_create()`, qui insère chaque niveau en une requête. Sous PostgreSQL, `bulk_create()` renvoie les clés primaires créées : les destinations copiées servent directement de parents aux activités copiées.

Extrait 6 – Clonage transactionnel (`apps/trips/services.py`, extrait).

```python
@transaction.atomic
def clone_trip(source: Trip, *, owner, title=None) -> Trip:
    trip = create_trip(owner=owner, title=title or f"Copie de {source.title}", ...,
                       status=Trip.Status.DRAFT)
    destinations = list(source.destinations.prefetch_related("activities"))
    # bulk_create renvoie les clés primaires sous PostgreSQL : les copies servent de parents.
    copies = Destination.objects.bulk_create(_copy(d, trip=trip) for d in destinations)
    Activity.objects.bulk_create(
        _copy(activity, destination=copy)
        for original, copy in zip(destinations, copies, strict=True)
        for activity in original.activities.all()
    )
    return trip
```

Participants. Le propriétaire ajoute un participant par son courriel, avec le rôle `editor` ou `viewer`. Le rôle `owner` ne s'attribue pas à l'invitation : il se transfère ensuite par un changement de rôle. L'ancien propriétaire devient alors éditeur. Le propriétaire ne peut être ni retiré ni rétrogradé directement ; le voyage resterait sans propriétaire. Un ajout en double viole la contrainte `trips_participant_unique_trip_user`. Le service laisse la base trancher, ce qui élimine toute course entre deux ajouts simultanés.

5.5 Administration et données de démonstration

L'administration de Django (`/admin/`) affiche les voyages avec leurs participants et leurs destinations en ligne (figure 4). La commande `seed_demo` crée trois comptes et quatre voyages (Montréal, Paris, Tokyo, Dakar) qui couvrent chaque rôle. Elle est idempotente, et ses dates sont relatives au jour de l'exécution : un voyage terminé, un en cours, un planifié et un brouillon.

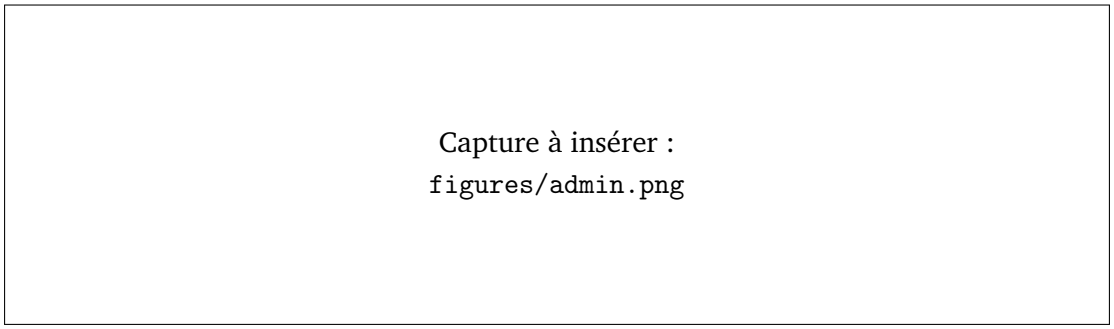


FIGURE 4 – Administration Django : fiche d'un voyage, avec ses participants et ses destinations en ligne.

6 Intégrations externes

TravelHub consomme deux API publiques, sans clé d'accès. Open-Meteo [12, 11] fournit le géocodage et la météo par une interface REST. Countries [1] décrit les pays par une interface GraphQL. Le code d'intégration est isolé dans `apps/integrations/services/`. Les clients renvoient des `dataclasses` immuables, jamais le JSON brut du fournisseur.

6.1 Open-Meteo (REST)

La météo d'une destination demande deux appels.

1. **Géocodage.** Une destination saisie sans coordonnées est d'abord localisée par `geocoding-api.open-meteo.com/v1/search`, à partir de la ville et du code pays. Les coordonnées trouvées sont enregistrées dans la destination : les appels suivants s'en passent.
2. **Prévisions.** L'appel `api.open-meteo.com/v1/forecast` renvoie les conditions actuelles et les prévisions quotidiennes des 16 prochains jours. Le service les filtre sur les dates du séjour.

Les codes météo WMO sont traduits en libellés français (« Pluie modérée », « Orage »…). Un séjour entièrement hors de la fenêtre de 16 jours produit l'erreur `no_data_available`. Une marge d'un jour absorbe l'écart entre la date du serveur et la date locale de la destination.
L'enregistrement des coordonnées utilise `update()` filtré sur `latitude IS NULL`, et non `save()`. Cette écriture ne touche que les deux colonnes concernées. Elle n'écrase pas une modification concurrente de la destination.

6.2 Countries (GraphQL)

Le client Countries envoie une requête GraphQL par `POST` à `countries.trevorblades.com/graphql`. Le code pays passe dans `variables`, jamais par concaténation dans le texte de la requête. Cette pratique ferme la porte à l'injection, de la même manière qu'une requête SQL paramétrée.

Extrait 7 – Requête GraphQL paramétrée (`apps/integrations/services/countries.py`, extrait condensé).

```
COUNTRY_QUERY = """
query Country($code: ID!) {
  country(code: $code) {
    code name native capital emoji currency
    continent { name }
    languages { name native }
  }
}
"""
body = {"query": COUNTRY_QUERY, "variables": {"code": code}}
```

Un serveur GraphQL répond généralement 200, même en cas d'erreur. Le client examine donc la liste `errors` avant le champ `data`. Une liste non vide donne une erreur 502. Un code inconnu donne `{"country": null}`, traduit en `country_not_found` (404).

6.3 Couche HTTP commune

Les deux clients partagent `apps/integrations/services/http.py`. Chaque requête a un délai de 5 secondes et dispose de deux essais. Le second essai n'a lieu que si l'échec est transitoire.

TABLE 7 – Politique de nouvel essai et traduction des échecs.

Situation	Nouvel essai	Exception	HTTP
Délai dépassé	oui	`ExternalServiceTimeout`	504
Erreur réseau	oui	`ExternalServiceUnavailable`	503
Réponse 5xx	oui	`ExternalServiceUnavailable`	503
Réponse 4xx	non	`ExternalServiceUnavailable`	503
JSON illisible ou de forme inattendue	non	`ExternalServiceBadResponse`	502
Ville ou pays inconnu	non	`LocationNotFound`, `CountryNotFound`	404
Séjour hors des 16 jours	non	`NoDataAvailable`	404

Une réponse 4xx n'est pas retentée : la requête serait refusée de nouveau, à l'identique. Les exceptions de `apps/integrations/exceptions.py` ne dépendent pas de DRF. Elles portent un statut HTTP et un code stable. Les API REST et GraphQL les traduisent chacune à leur façon. Chaque essai est journalisé avec le service, l'URL sans paramètres, le statut, la durée et le numéro d'essai.

6.4 Cache

Les réponses sont conservées dans le cache de Django. Seule une réponse jugée exploitable par l'analyseur est mise en cache : une erreur transitoire ne reste pas « collée » pendant toute la durée de vie de l'entrée.

TABLE 8 – Durées de mise en cache.

Donnée	Durée	Clé
Géocodage	7 jours	code pays et condensat SHA-256 du nom de ville
Prévisions	30 minutes	position arrondie à 4 décimales (environ 10 m)
Pays	24 heures	code pays

Le condensat du nom de ville garantit une clé valide pour tout moteur de cache, quels que soient les caractères saisis. L'arrondi de la position permet à des destinations voisines de partager une entrée.

6.5 Agrégation et dégradation partielle

L'action `GET /api/destinations/{id}/overview/` réunit en une réponse la destination, son voyage, ses activités, le coût total, la météo et le pays. Les deux services externes sont interrogés en parallèle par un `ThreadPoolExecutor`, pendant que le fil principal lit la base.

Extrait 8 – Appels parallèles (`apps/integrations/services/aggregator.py`, extrait).

```python
with ThreadPoolExecutor(max_workers=len(SOURCES), thread_name_prefix="overview") as pool:
    futures = {
        WEATHER: pool.submit(contextvars.copy_context().run, fetch_forecast, destination),
        COUNTRY: pool.submit(
            contextvars.copy_context().run, countries.get_country, destination.country_code
        ),
    }
    activities = list(destination.activities.all())  # pendant les appels externes
    results, warnings = {}, []
    for source, future in futures.items():
        try:
            results[source] = future.result()
        except IntegrationError as exc:
            results[source] = None
            warnings.append(SourceWarning(source, exc.code, exc.message))
```

Trois choix encadrent ce parallélisme.

— Les fils secondaires n'exécutent aucune requête SQL. Chacun ouvrirait sa propre connexion, jamais fermée, et ne verrait pas la transaction du fil principal. Le géocodage calcule les coordonnées sans les écrire ; le fil principal les enregistre ensuite.

— Chaque tâche s'exécute dans une copie du contexte (`contextvars`). Ses journaux portent ainsi l'identifiant de la requête en cours.

— Une source en échec ne fait pas échouer la réponse, qui reste un 200. Son bloc vaut `null`, et la liste `warnings` explique pourquoi (extrait 9).

Extrait 9 – Dégradation partielle : Open-Meteo indisponible.

```
"weather": null,
"warnings": [
  {"source": "weather", "code": "external_service_unavailable",
   "message": "Open-Meteo est momentanément indisponible (erreur 503)."}
]
```

Cette dégradation partielle sert directement le futur front-end : la page d'une destination reste affichable quand un fournisseur tombe.

7 API REST

7.1 Points d'accès

L'API REST est construite avec Django REST Framework [2]. Les tableaux 9 à 11 recensent ses points d'accès. Tous exigent un jeton d'accès (`Authorization: Bearer <access>`), sauf l'inscription, la connexion, le rafraîchissement et la documentation.

TABLE 9 – Authentification (`apps/accounts/urls.py`).

Méthode	Chemin	Rôle
POST	`/api/auth/register/`	Inscription ; renvoie l'utilisateur et une paire de jetons
POST	`/api/auth/token/`	Connexion ; jeton d'accès (15 min) et de rafraîchissement (1 jour)
POST	`/api/auth/token/refresh/`	Nouveau jeton d'accès ; l'ancien jeton de rafraîchissement est révoqué
POST	`/api/auth/logout/`	Révocation de son propre jeton de rafraîchissement
GET	`/api/auth/me/`	Profil de l'utilisateur connecté

TABLE 10 – Voyages et participants (`apps/trips/urls.py`).

Méthode	Chemin	Rôle
GET, POST	`/api/trips/`	Mes voyages (paginés par 20) ; création
GET, PUT, PATCH, DELETE	`/api/trips/{id}/`	Détail avec destinations et activités ; modification ; suppression
GET	`/api/trips/{id}/stats/`	Coût total, budget restant, coûts par catégorie
POST	`/api/trips/{id}/clone/`	Copie du voyage en brouillon, en une transaction
GET, POST	`/api/trips/{id}/participants/`	Liste ; ajout par courriel
PATCH, DELETE	`/api/trips/{id}/participants/{user_id}/`	Changement de rôle ; retrait

TABLE 11 – Destinations et activités (`apps/itinerary/urls.py`).

Méthode	Chemin	Rôle
GET, POST	`/api/destinations/`	Destinations de mes voyages ; création
GET, PUT, PATCH, DELETE	`/api/destinations/{id}/`	Détail ; modification ; suppression
GET	`/api/destinations/{id}/weather/`	Météo du séjour (Open-Meteo)
GET	`/api/destinations/{id}/country/`	Fiche du pays (Countries)
GET	`/api/destinations/{id}/overview/`	Vue d'ensemble, avec dégradation partielle
GET, POST	`/api/activities/`	Activités de mes voyages ; création
GET, PUT, PATCH, DELETE	`/api/activities/{id}/`	Détail ; modification ; suppression

S'y ajoutent `/api/schema/` (schéma OpenAPI), `/api/docs/` (Swagger UI), `/api/redoc/` (ReDoc), `/graphql/` et `/health/`.

7.2 Serializers

Les serializers convertissent les modèles en JSON et valident les entrées. Ils appliquent une liste blanche de champs. Les champs calculés ou sensibles sont en lecture seule : `owner`, `id`, horodatages et totaux. Un client ne peut donc pas s'attribuer un voyage en envoyant `"owner": 7`.

Trois variantes servent le voyage. `TripSerializer` sert à la liste, à la création et à la modification. `TripDetailSerializer` y ajoute les destinations et leurs activités, imbriquées en lecture seule. Des serializers sans modèle décrivent les réponses calculées, comme `TripStatsSerializer`. Le mixin `ModelCleanMixin`, présenté en section 9, applique en plus `Model.clean()` et les contraintes du modèle.

7.3 ViewSets et routage

Chaque ressource est un `ModelViewSet`, enregistré auprès d'un `DefaultRouter`. Le routeur génère les routes de liste et de détail. Les opérations hors CRUD sont des actions déclarées par `@action` : `stats`, `clone`, `participants`, `weather`, `country` et `overview`.

Extrait 10 – Queryset et permissions du ViewSet des voyages (`apps/trips/views.py`, extrait).

```python
class TripViewSet(TripRolePermissionMixin, viewsets.ModelViewSet):
    filterset_class = TripFilter
    search_fields = ["title", "description"]
    ordering_fields = ["start_date", "title", "created_at"]
    object_permission_by_action = {
        "update": IsTripEditor,
        "partial_update": IsTripEditor,
        "destroy": IsTripOwner,
        "participant_detail": IsTripOwner,
    }

    def get_queryset(self):
        queryset = Trip.objects.visible_to(self.request.user).with_totals().select_related("owner")
        if self.action == "retrieve":
            queryset = queryset.prefetch_related(
                Prefetch("destinations",
                         queryset=Destination.objects.order_by("order").prefetch_related("activities"))
            )
        return queryset
```

Le dictionnaire `object_permission_by_action` associe chaque action à sa permission objet. Une action absente, en pratique une lecture, exige seulement d'être membre du voyage. Les règles d'accès se lisent ainsi en un coup d'œil.

7.4 Filtres, recherche, tri et pagination

Trois moteurs de filtrage sont actifs par défaut : `django-filter`, la recherche plein texte de DRF et un tri stable, qui départage toujours par identifiant. La pagination est fixée à 20 éléments par page.

TABLE 12 – Paramètres des listes.

Ressource	Filtres	Tri (`ordering`)
Voyages	`status`, `start_date__gte`, `end_date__lte`, `title__icontains`, `search`	date, titre, création
Destinations	`trip`, `country_code`, `arrival_date__gte`, `departure_date__lte`, `search`	ordre, arrivée, ville
Activités	`trip`, `destination`, `category`, `date`, `date__gte`, `date__lte`, `search`	coût, date, nom

Les filtres `?trip=` et `?destination=` sont de simples filtres numériques appliqués au queryset déjà restreint. Un identifiant étranger donne une liste vide, et non une erreur : le filtre ne révèle pas l'existence des voyages des autres.

7.5 Codes HTTP et format des erreurs

L'API emploie les codes HTTP selon leur sémantique (tableau 13).

TABLE 13 – Codes HTTP renvoyés.

Code	Code d'erreur	Situation
200, 201, 204	—	Lecture ou modification, création, suppression
400	`validation_error`, `integrity_error`	Données invalides, contrainte violée
401	`not_authenticated`, `token_not_valid`	Jeton absent, invalide ou expiré
403	`permission_denied`	Membre du voyage sans le rôle requis
404	`not_found`	Ressource absente ou appartenant à autrui
405, 415	`method_not_allowed`, `unsupported_media_type`	Méthode ou type de contenu refusé
429	`throttled`	Limite de requêtes atteinte
502, 503, 504	`external_service_*`	Service externe illisible, indisponible ou trop lent

Toutes les erreurs partagent la même enveloppe. Le champ `code` est un identifiant stable, destiné au programme client. Le champ `message` s'adresse à l'humain. Le champ `details` donne les erreurs par champ.

Extrait 11 – Enveloppe d'erreur d'une validation.

```
{
  "error": {
    "code": "validation_error",
    "message": "Les données envoyées sont invalides.",
    "details": {"end_date": ["La date de fin doit être postérieure ou égale à la date de début."]}
  }
}
```

Le gestionnaire `apps.core.exceptions.exception_handler` produit cette forme. Il convertit aussi les exceptions qui, sans lui, donneraient une erreur 500 : `ValidationError` de Django, levée par les services, et `IntegrityError`, levée par PostgreSQL. Les URL inconnues et les erreurs inattendues hors DRF (`handler404`, `handler500`) renvoient la même enveloppe, sans page HTML ni trace de pile.

7.6 Documentation OpenAPI

La bibliothèque drf-spectacular [5] génère un schéma OpenAPI 3 à partir des ViewSets et des serializers. Chaque opération est annotée par `@extend_schema` : résumé en

français, réponses d'erreur possibles et exemples de requête et de réponse. L'option `COMPONENT_SPLIT_REQUEST` sépare les schémas de lecture et d'écriture : les champs calculés n'apparaissent pas dans les corps de requête. Swagger UI (figure 5) permet de tester l'API après avoir collé un jeton dans *Authorize*.

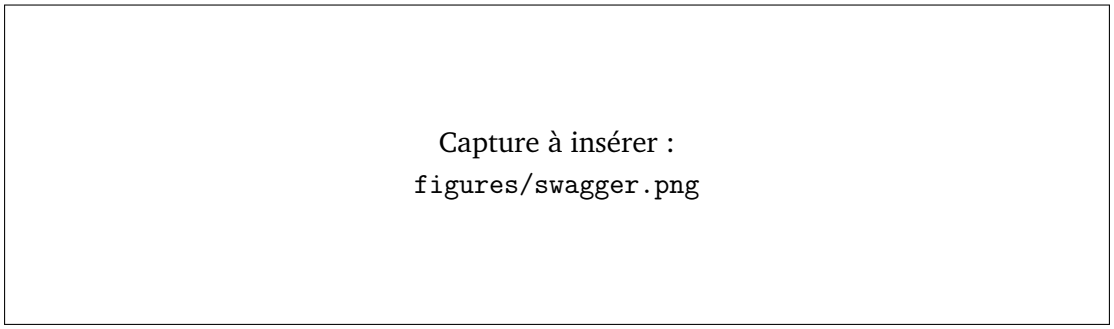


FIGURE 5 – Swagger UI (`/api/docs/`) : opérations sur les voyages, regroupées par étiquette.

8 API GraphQL

8.1 Schéma

L'API GraphQL est construite avec Graphene-Django [7] et suit la spécification GraphQL [8]. Elle expose un point d'entrée unique, `POST /graphql/`. Chaque application déclare ses types, ses requêtes et ses mutations dans son propre `schema.py`. Le schéma racine les assemble par héritage multiple.

Extrait 12 – Assemblage du schéma (`config/schema.py`).

```
class Query(itinerary.Query, trips.Query, accounts.Query, graphene.ObjectType):
    pass


class Mutation(itinerary.Mutation, trips.Mutation, graphene.ObjectType):
    pass


schema = graphene.Schema(query=Query, mutation=Mutation)
```

Les types `User`, `Trip`, `TripParticipant`, `Destination` et `Activity` dérivent de `DjangoObjectType`. Chacun déclare une liste blanche de champs : le type `User` n'expose ni mot de passe ni droits d'administration, et le courriel n'est visible que par son titulaire. Les choix des modèles deviennent des énumérations GraphQL (`TripStatus`, `ParticipantRole`, `ActivityCategory`). Les données externes ont leurs propres types : `Weather`, `CountryInfo` et `DestinationOverview`.

TABLE 14 – Requêtes et mutations.

Racine	Champs
Query	`me`, `myTrips(status, startAfter, search, orderBy, limit, offset)`, `myTripsCount`, `trip(id)`, `destination(id)`, `destinationOverview(id)`
Mutation	`createTrip`, `updateTrip`, `deleteTrip`, `cloneTrip`, `addParticipant`, `removeParticipant`, `createDestination`, `updateDestination`, `deleteDestination`, `createActivity`, `updateActivity`, `deleteActivity`

8.2 Réutilisation de la logique REST

L'API GraphQL ne duplique aucune règle. Tout resolver part de `Trip.objects.visible_to(user)`. Les mutations réutilisent les serializers DRF, donc `Model.clean()` et les contraintes, ainsi que les services (`create_trip`, `clone_trip`, `add_participant`). Les permissions objet sont les mêmes classes que côté REST. Chaque mutation s'exécute dans une transaction (`@transaction.atomic`).

Extrait 13 – Mutation de modification (`apps/trips/schema.py`, extrait).

```python
class UpdateTrip(graphene.Mutation):
    class Arguments:
        id = graphene.ID(required=True)
        input = TripUpdateInput(required=True)

    trip = graphene.Field(TripType)

    @classmethod
    @transaction.atomic
    def mutate(cls, root, info, id, input):
        trip = get_or_not_found(visible_trips(info), TRIP_NOT_FOUND, pk=id)
        check_object_permission(info, IsTripEditor, trip)
        serializer = TripSerializer(trip, data=dict(input), partial=True,
                                    context=serializer_context(info))
        return cls(trip=reload_trip(info, save_serializer(serializer)))
```

8.3 Authentification, limites et erreurs

La vue `TravelHubGraphQLView` hérite de `GraphQLView`. Elle authentifie le jeton avec la classe `JWTAuthentication` de SimpleJWT, comme l'API REST. Sans en-tête, la requête est anonyme et chaque champ protégé répond `not_authenticated`. Un jeton invalide ou expiré donne un 401. Le cookie de session n'est jamais pris en compte, ce qui permet d'exempter la vue de la protection CSRF sans risque. Les limites de requêtes reprennent les classes et les compteurs de DRF : un utilisateur dispose d'un seul quota, partagé entre REST et GraphQL.

Les erreurs conservent le code et les détails de l'erreur REST équivalente, dans le champ `extensions`. Une erreur inattendue ne divulgue pas son message : elle devient `internal_error`, et sa trace est journalisée.

Extrait 14 – Erreur de validation d'une mutation `updateTrip`.

```
{
  "errors": [{
    "message": "Les données envoyées sont invalides.",
    "path": ["updateTrip"],
    "extensions": {
      "code": "validation_error",
      "details": {"endDate": ["La date de fin doit être postérieure ou égale à la date de début."]}
    }
  }],
  "data": {"updateTrip": null}
}
```

8.4 Durcissement et performance

Trois paramètres, relus à chaque requête, durcissent le point d'entrée.

— `GRAPHQL_MAX_DEPTH = 6` : la profondeur des requêtes est limitée. Les requêtes cycliques (`trip { destinations { trip { ... } } }`) sont rejetées avant leur exécution.
— `GRAPHQL_INTROSPECTION` : l'introspection (`__schema`, `__type`) est fermée en production. Le champ `__typename`, ajouté par les clients comme Apollo, reste permis.
— `GRAPHQL_GRAPHIQL` : l'interface GraphiQL n'est servie qu'en développement.

Côté performance, `myTrips` et `trip` n'activent le préchargement des destinations, des activités ou des participants que si ces champs figurent dans la requête. La fonction `selected_fields()` parcourt la sélection, fragments compris. Le nombre de requêtes SQL ne dépend donc pas du nombre de voyages renvoyés. La liste `myTrips` est paginée comme l'API REST : 20 éléments par défaut, 100 au plus.

8.5 Exemple

L'extrait 15 obtient, en un seul aller-retour, les voyages planifiés, leurs destinations, la météo actuelle et la fiche de chaque pays. En REST, la même information exigerait une requête pour la liste, puis deux requêtes par destination.

Extrait 15 – Requête GraphQL agrégée.

```
query MesVoyagesPlanifies {
  myTripsCount(status: PLANNED)
  myTrips(status: PLANNED, orderBy: START_DATE, limit: 5) {
    id
    title
    startDate
    totalCost
    destinations {
      city
      weather { current { temperature description } }
      countryInfo { name emoji currencies }
    }
  }
}
```

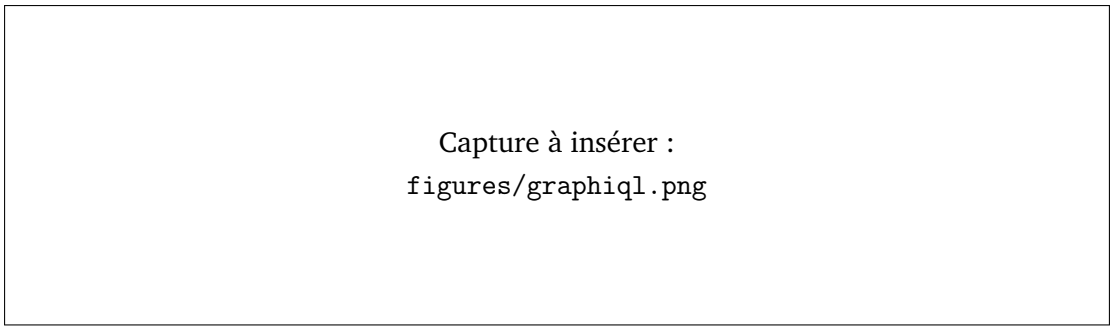


FIGURE 6 – GraphiQL (`/graphql/`, développement) : requête `myTrips` authentifiée par l'entête `Authorization`.

8.6 Comparaison REST et GraphQL

Le tableau 15 compare les deux API telles qu'elles sont réalisées dans TravelHub. REST, au sens de Fielding [6], organise l'API autour de ressources adressables. GraphQL l'organise autour d'un schéma typé que le client interroge.

TABLE 15 – Comparaison des deux API de TravelHub.

Critère	REST (DRF)	GraphQL (Graphene)
Points d'entrée	Une URL par ressource et par action (18 chemins)	Un seul : `POST /graphql/`
Forme de la réponse	Fixée par le serveur (serializer)	Choisie par le client, champ par champ
Sur- et sous-récupération	Liste sans détail ; vue d'ensemble dédiée (`overview`) pour éviter les allers-retours	Évitées : le client demande exactement les champs voulus
Signalement des erreurs	Code HTTP (400, 403, 404…) et enveloppe `error`	HTTP 200 et liste `errors`, avec `extensions.code` ; 400 pour une requête invalide
Échec partiel	Tout ou rien, sauf `overview` (`warnings`)	Natif : un champ en échec vaut `null`, les autres sont servis
Cache HTTP	Possible sur les `GET` (URL stables)	Difficile : tout passe par `POST`
Documentation	OpenAPI, Swagger UI, ReDoc	Schéma typé, GraphiQL ; introspection fermée en production
Pagination	Numéro de page (`?page=`)	`limit` et `offset`, plus `myTripsCount`
Risques propres	Exposition de champs en trop (affectation de masse)	Requêtes coûteuses : profondeur limitée à 6, largeur non limitée
Authentification, permissions	JWT, `visible_to()`, classes de `apps.core.permissions`	Identiques ; quota de requêtes partagé avec REST
Validation	Serializers, `Model.clean()`, contraintes	Identique : mêmes serializers appelés par les mutations

Les deux API ne s'opposent pas ; elles se complètent. REST convient aux opérations simples et aux intégrations tierces, qui profitent du cache HTTP et d'OpenAPI. GraphQL convient aux écrans riches du futur front-end, qui assemblent plusieurs ressources en une requête. Le coût de cette double exposition reste faible, car toute la logique est partagée.

9 Validation, transactions et intégrité

9.1 Trois niveaux de validation

Une donnée franchit trois contrôles successifs avant d'être écrite (tableau 16). Chaque niveau a un rôle propre ; aucun ne suffit seul.

TABLE 16 – Niveaux de validation.

Niveau	Règles	Raison d'être
Serializer	Types, longueurs, choix, champs en lecture seule ; cohérence des dates en `PATCH`	Refuser tôt, avec un message par champ
`Model.clean()`	Destination dans les dates du voyage ; activité dans les dates de la destination ; pas de raccourcissement qui laisserait un enfant hors période	Règles métier qui lisent d'autres lignes
Contraintes PostgreSQL	`CHECK` et `UNIQUE` du tableau 5	Dernier rempart, valable même en cas d'accès concurrent

En `PATCH`, une seule des deux dates peut être envoyée. Le serializer complète alors la paire avec la valeur enregistrée avant de la comparer. Sans cette précaution, une date de fin antérieure à la date de début existante passerait la validation et échouerait seulement en base.

9.2 Appliquer `Model.clean()` depuis DRF

DRF n'appelle ni `Model.clean()` ni `validate_constraints()`. Sans intervention, une règle de `clean()` serait ignorée par l'API, et une contrainte violée produirait une `IntegrityError` au moment de l'écriture. Le mixin `ModelCleanMixin` comble ce manque.

Extrait 16 – Validation par le modèle (`apps/core/serializers.py`, extrait).

```python
class ModelCleanMixin:
    def validate(self, attrs):
        attrs = super().validate(attrs)
        if self.instance is not None:
            candidate = copy.copy(self.instance)
        else:
            candidate = self.Meta.model()
        for field, value in attrs.items():
            setattr(candidate, field, value)
        try:
            candidate.clean()
            candidate.validate_constraints()
        except DjangoValidationError as exc:
            raise serializers.ValidationError(django_error_detail(exc)) from exc
        return attrs
```

Les règles sont évaluées sur une *copie* de l'instance. Si la validation échoue, l'objet chargé par la vue reste intact. Les messages de Django sont ramenés au format de DRF : la clé `__all__` devient `non_field_errors`.

9.3 Contraintes violées malgré la validation

Deux requêtes simultanées peuvent passer la validation, puis entrer en conflit à l'écriture. Par exemple, deux éditeurs placent chacun une destination en position 2. La contrainte `itinerary_destination_unique_trip_order` rejette la seconde écriture. Le gestionnaire d'exceptions convertit alors l'`IntegrityError` en réponse 400. Il retrouve le message de la contrainte grâce au nom que PostgreSQL renvoie dans son diagnostic.

Extrait 17 – Message de la contrainte violée (`apps/core/exceptions.py`, extrait).

```python
def integrity_error_message(exc):
    """Message de la contrainte violée, retrouvé grâce au nom que remonte PostgreSQL."""
    diag = getattr(exc.__cause__, "diag", None)  # psycopg expose le diagnostic du serveur
    name = getattr(diag, "constraint_name", None)
    return _constraint_messages().get(name, INTEGRITY_MESSAGE)
```

Le client reçoit ainsi « Une destination occupe déjà cette position dans le voyage. », avec le code `integrity_error`, au lieu d'une erreur 500.

9.4 Transactions

Les opérations qui écrivent plusieurs lignes sont atomiques. Les services `create_trip`, `clone_trip` et `change_participant_role` sont décorés par `@transaction.atomic`, de même que chaque mutation GraphQL. Le service `add_participant` ouvre un point de sauvegarde (`transaction.atomic()` imbriqué) autour de l'insertion : une violation d'unicité y est rattrapée sans rendre inutilisable la transaction englobante.

9.5 Démonstration du rollback

Le clonage d'un voyage écrit dans quatre tables : voyage, participant, destinations, puis activités. Le test de l'extrait 18 provoque une panne au dernier moment, pendant la copie des activités. Il compare ensuite le contenu des quatre tables avant et après la requête.

Extrait 18 – Panne simulée pendant le clonage (`test_trip_actions.py`, extrait).

```
def test_failure_midway_gives_500_and_leaves_database_unchanged(
    self, client_for, owner, source, monkeypatch
):
    def snapshot():
        return [
            list(model.objects.order_by("pk").values())
            for model in (Trip, TripParticipant, Destination, Activity)
        ]

    def failing_bulk_create(objs, *args, **kwargs):
        raise RuntimeError("panne simulée pendant la copie des activités")

    before = snapshot()
    monkeypatch.setattr(Activity.objects, "bulk_create", failing_bulk_create)
    client = client_for(owner)
    client.raise_request_exception = False

    response = client.post(clone_url(source))

    assert response.status_code == 500
    assert response.json() == {"error": INTERNAL_ERROR}
    assert snapshot() == before
```

Au moment de la panne, le nouveau voyage, son participant et ses destinations ont déjà été insérés. La transaction les annule tous. Le client reçoit une erreur 500 au format JSON, sans trace de pile, et la base est identique à son état initial. Le même scénario est vérifié pour la mutation GraphQL `cloneTrip` (`apps/trips/tests/test_graphql.py`) et pour le service seul (`test_services.py`).

10 Sécurité

10.1 Authentification par JWT

L'API est sans état : chaque requête porte un jeton JWT [10], émis par SimpleJWT [9]. Le jeton d'accès expire après 15 minutes, ce qui limite l'effet d'un vol. Le jeton de rafraîchissement dure un jour et ne sert qu'une fois : chaque rafraîchissement en émet un nouveau et place l'ancien sur une liste noire. La déconnexion révoque le jeton fourni, à condition qu'il appartienne à l'utilisateur connecté.

Extrait 19 – Paramètres JWT (`config/settings/base.py`).

```
SIMPLE_JWT = {
    "ACCESS_TOKEN_LIFETIME": timedelta(minutes=15),
    "REFRESH_TOKEN_LIFETIME": timedelta(days=1),
    # Chaque rafraîchissement émet un nouveau refresh et met l'ancien sur liste noire.
    "ROTATE_REFRESH_TOKENS": True,
    "BLACKLIST_AFTER_ROTATION": True,
    "UPDATE_LAST_LOGIN": True,
    "AUTH_HEADER_TYPES": ("Bearer",),
}
```

Les mots de passe passent les quatre validateurs de Django : longueur minimale, similarité avec le profil, mots de passe courants, valeurs entièrement numériques.

10.2 Permissions par rôle

DRF refuse par défaut toute requête anonyme (`IsAuthenticated`). Trois permissions objet (`apps/core/permissions.py`) traduisent ensuite les rôles : `IsTripMember`, `IsTripEditor` et `IsTripOwner`. Elles acceptent un voyage ou tout objet qui s'y rattache ; une activité remonte à son voyage par `activity.destination.trip`.

TABLE 17 – Matrice d'accès à un voyage (codes HTTP).

Opération	Propriétaire	Éditeur	Lecteur	Non participant
Lire le voyage, `stats`, `clone`	200 / 201	200 / 201	200 / 201	404
Modifier le voyage, ses destinations, ses activités	200	200	403	404
Supprimer le voyage	204	403	403	404
Gérer les participants	201 / 200 / 204	403	403	404

10.3 Contrôle d'accès au niveau objet

Le risque principal d'une API est le contrôle d'accès défaillant au niveau objet (*Broken Object Level Authorization*, API1 de l'OWASP [13]) : modifier l'identifiant dans l'URL pour atteindre l'objet d'autrui. TravelHub s'en protège par construction. Chaque vue, chaque resolver et chaque filtre part de `visible_to(user)`. Un voyage d'autrui est absent du queryset, et la réponse est 404. Le même principe s'applique au parent d'une création : une destination créée sur le voyage d'autrui donne 404, sur un voyage visible sans droit d'écriture 403.

10.4 Configuration de production

Le module `config.settings.prod` active les protections du transport et du navigateur : redirection HTTPS, HSTS de 30 jours, cookies `Secure`, `X-Frame-Options: DENY`, `nosniff`, politique de référent et COOP. L'API navigable de DRF y est désactivée : seul le JSON est servi. GraphiQL et l'introspection y sont fermés. La commande suivante, exécutée par un test, ne produit aucun avertissement [3] :

Extrait 20 – Vérification de la configuration de production.

```
python manage.py check --deploy --settings=config.settings.prod
```

CORS. Seules les origines listées dans `CORS_ALLOWED_ORIGINS` sont acceptées, sans transmission des identifiants. La liste est vide par défaut ; le TP2 y ajoutera l'origine du front-end.

Secrets. La clé secrète et l'accès à la base sont lus dans l'environnement. Le fichier `.env` est ignoré par Git ; `.env.example` ne contient que des valeurs fictives. Un test vérifie que `SECRET_KEY` provient bien de l'environnement. Les journaux ne contiennent ni corps de requête, ni en-têtes, ni chaîne de requête. En filet de sécurité, le formateur masque tout champ dont le nom évoque un secret, ainsi que tout JWT présent dans un message.

10.5 OWASP API Security Top 10

Le tableau 18 résume la couverture des dix risques de l'édition 2023.

TABLE 18 – Mesures face à l'OWASP API Security Top 10 (2023).

Risque	Mesures
API1 Object Level Authorization	`visible_to()` partout ; 404 plutôt que 403 ; parent cherché parmi les objets visibles
API2 Authentication	Accès de 15 min, rotation et liste noire, validateurs de mot de passe, cookie de session ignoré par GraphQL
API3 Object Property Level Authorization	Listes blanches de champs ; `owner`, totaux et horodatages en lecture seule ; pas de `is_staff` à l'inscription
API4 Resource Consumption	100 requêtes/h en anonyme, 1000/h par utilisateur ; pagination ; profondeur GraphQL limitée ; délai de 5 s vers l'extérieur
API5 Function Level Authorization	Permissions par rôle ; seul le propriétaire gère les participants
API6 Sensitive Business Flows	Inscription, connexion et clonage soumis aux limites de requêtes
API7 Server Side Request Forgery	URL externes constantes ; ville en paramètre, code pays en variable GraphQL
API8 Security Misconfiguration	`check -deploy` propre ; en-têtes de sécurité ; erreurs JSON sans trace
API9 Inventory Management	Une seule version, décrite par OpenAPI et par la collection Postman
API10 Unsafe Consumption of APIs	Réponses externes validées et converties en `dataclasses` ; erreurs 502, 503, 504

Chaque ligne est couverte par des tests dédiés. La matrice d'accès croisés de `test_object_level_authorization.py` vérifie, pour chaque point d'accès REST et chaque opération GraphQL, qu'un non-participant, un lecteur et un anonyme sont refusés, et que la base reste inchangée après le refus.

10.6 Risques résiduels

Trois risques sont connus et acceptés.

— L'inscription et l'ajout d'un participant indiquent si un courriel est inscrit. L'énumération reste possible, freinée par les limites de requêtes.
— GraphQL limite la profondeur des requêtes, mais pas leur largeur (alias multiples).
— La documentation OpenAPI reste publique en production, alors que l'introspection GraphQL y est fermée.

11 Tests

11.1 Stratégie

La suite repose sur pytest et pytest-django. Elle s'exécute sur une vraie base PostgreSQL, créée et détruite à chaque exécution (`test_travelhub`). SQLite aurait été plus rapide, mais il ignore l'index unique partiel et certaines contraintes `CHECK` : les tests d'intégrité n'auraient rien prouvé. Aucun test ne sort sur le réseau. La fixture `mock_http`, fondée sur la bibliothèque `responses`, intercepte Open-Meteo et Countries et permet de simuler leurs pannes.

Les fixtures communes (`conftest.py`) rendent les tests courts et lisibles :

— deux utilisateurs, `user_a` et `user_b`, avec leurs objets et leurs clients authentifiés, pour les accès croisés ;
— des fabriques (`make_trip`, `make_destination`…) qui produisent des objets valides, dont chaque champ peut être remplacé ;
— une fixture `graphql(query, variables, user=...)` qui envoie une requête GraphQL avec un vrai jeton.

11.2 Couverture fonctionnelle

TABLE 19 – Scénarios vérifiés.

Domaine	Cas	Attendu
Authentification	Sans jeton ; jeton falsifié, non signé ou expiré ; refresh révoqué	401
Voyages	Création ; fin avant début ; titre vide ; filtres ; tri	201 / 400 / 200
Accès croisés	B lit, modifie ou supprime le voyage de A	404
Rôles	Lecteur qui modifie ; éditeur qui supprime ou invite	403
Destinations	Dates hors voyage ; position dupliquée ; voyage tiers	400 / 404
Activités	Coût négatif ; date hors séjour ; filtre par catégorie	400 / 200
Transaction	Panne forcée pendant le clonage	500, base inchangée
Open-Meteo	Ville inconnue ; service en 500 ; délai dépassé	404 / 503 / 504
Countries	Code inconnu ; liste `errors` non vide	404 / 502
Agrégation	Une source en panne	200 et `warnings`
GraphQL	Sans jeton ; mutation sur un voyage tiers ; validation ; profondeur	`extensions.code`
Sécurité	Affectation de masse ; en-têtes ; journaux sans secret ; `check -deploy`	—

11.3 Résultats et couverture

La suite compte **724 tests**, tous réussis (exécution du 23 octobre 2026, environ deux minutes et demie). La couverture, mesurée sur `apps/` et `config/` hors tests et migrations, atteint **98 %** : 2 139 instructions et 242 branches, dont 16 instructions non exécutées. L'intégration continue échoue si la couverture totale passe sous 80 % (`-cov-fail-under=80`). Le seuil n'est pas un objectif en soi : les lignes restantes sont surtout des chemins défensifs.

Capture à insérer :
`figures/couverture.png`

FIGURE 7 – Rapport de couverture HTML (`htmlcov/index.html`).

11.4 Collection Postman

La collection `postman/TravelHub.postman_collection.json` (format v2.1) rejoue un parcours complet sur un serveur réel : 85 requêtes réparties en neuf dossiers (tableau 20). Chaque requête porte des assertions sur le statut et le corps de la réponse. Le script de la requête de connexion enregistre les jetons dans l'environnement ; l'autorisation *Bearer* `{{access}}` est héritée par toutes les requêtes.

TABLE 20 – Dossiers de la collection Postman.

Dossier	Requêtes	Contenu
Auth	6	Inscription de deux comptes, connexion, rotation, profil, déconnexion
Trips	9	CRUD, recherche, filtre, tri, statistiques, duplication
Destinations	6	CRUD et filtres
Activities	7	CRUD, filtre par catégorie, statistiques après ajout
Participants	10	Lecteur puis éditeur refusés en écriture (403), retrait (404)
Externes	3	Météo, pays, vue d'ensemble
GraphQL	9	`me`, `myTrips`, `trip`, mutations, `destinationOverview`
Cas d'erreur	33	400, 401, 404, 405, 415, affectation de masse, erreurs GraphQL
Nettoyage	2	Suppression des données de l'exécution

Chaque exécution crée des comptes uniques et des dates relatives au jour même : la collection peut être relancée sans préparation. Elle s'exécute dans le *Runner* de Postman (figure 8) ou en ligne de commande :

Extrait 21 – Exécution de la collection en ligne de commande.

```
postman collection run postman/TravelHub.postman_collection.json -e
    postman/TravelHub.postman_environment.json
```

Capture à insérer :
`figures/postman-runner.png`

FIGURE 8 – Exécution de la collection dans le Runner de Postman : toutes les assertions réussies.

Les deux outils se complètent. Pytest simule les pannes impossibles à provoquer à la

demande : service externe en 500, délai dépassé, panne au milieu d'une transaction. Postman vérifie le comportement de bout en bout, sur le vrai serveur et les vrais services externes.

11.5 Qualité du code

Ruff assure l'analyse statique (`ruff check`) et le formatage (`ruff format`). Les deux vérifications s'exécutent dans l'intégration continue, avant les tests.

12 Industrialisation

Cette partie dépasse l'énoncé. Elle n'a été entreprise qu'une fois toutes les exigences satisfaites.

12.1 Docker

Le `Dockerfile` construit l'image en deux étapes. La première installe les dépendances dans un environnement virtuel, à partir de `python:3.14-slim`. La seconde copie cet environnement dans une image d'exécution sans outils de construction. L'image exécute `collectstatic` : WhiteNoise sert ensuite les fichiers statiques. Gunicorn tourne sous un utilisateur non privilégié, qui ne peut écrire que dans `/app/logs`. Une instruction `HEALTHCHECK` interroge `/health/` toutes les 30 secondes.
Le fichier `docker-compose.yml` assemble trois services (tableau 21).

TABLE 21 – Services Docker Compose.

Service	Rôle
`web`	Image du projet, paramètres de production ; applique les migrations puis lance Gunicorn ; démarre quand `db` est prêt
`db`	`postgres:18`, volume persistant, contrôle `pg_isready` ; port non publié
`pgadmin`	pgAdmin 4, accessible depuis le poste local seulement ; serveur TravelHub déjà déclaré

12.2 Intégration continue

Le flux GitHub Actions (`.github/workflows/ci.yml`) s'exécute à chaque *push* sur `master` et à chaque *pull request*. Il comporte deux tâches.

1. **Lint et tests.** Un service PostgreSQL 18 démarre, puis Ruff vérifie le code et son formatage. Pytest s'exécute ensuite avec le seuil de couverture de 80 %. Le résumé de couverture s'affiche dans la page de l'exécution, et les rapports sont publiés comme artefacts.

2. **Image Docker.** L'image est construite avec un cache, puis démarrée par `docker compose up -wait`. La tâche vérifie enfin que `/health/`, `/api/docs/` et `/api/schema/` répondent.

Dependabot propose chaque semaine les mises à jour des dépendances pip et des actions GitHub.

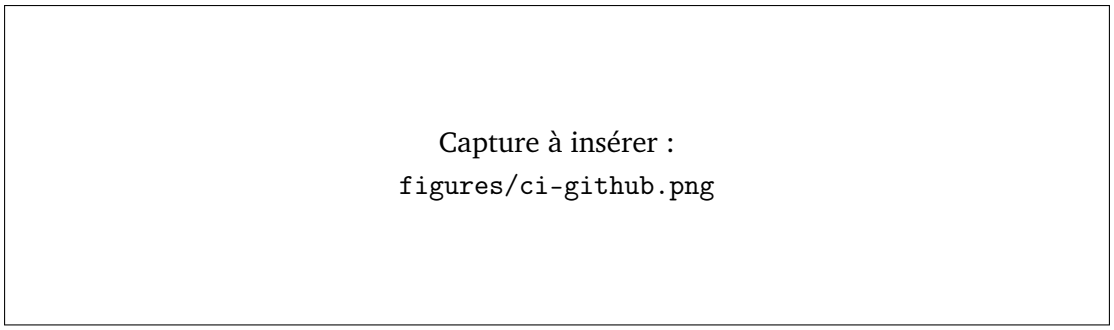


FIGURE 9 – Exécution réussie du flux d'intégration continue sur GitHub Actions.

12.3 Journaux et contrôle de santé

Chaque événement produit une ligne JSON, sur la console et dans un fichier rotatif (10 Mo, 5 archives). L'intergiciel `RequestIdMiddleware` attribue à chaque requête un identifiant, repris dans toutes ses lignes de journal et renvoyé dans l'en-tête `X-Request-ID`. Une erreur signalée par un utilisateur se retrouve ainsi en une recherche.

Extrait 22 – Ligne de journal d'une requête.

```
{"timestamp": "2026-10-23T22:01:39.510+00:00", "level": "INFO", "logger": "travelhub.request",
 "message": "GET /api/trips/ 200", "request_id": "292f04c8d5e541e69c07c7565e0e65f0",
 "method": "GET", "path": "/api/trips/", "status": 200, "duration_ms": 401.1, "user_id": 5}
```

Le point d'accès `/health/` vérifie la base (`SELECT 1`) et le cache (écriture puis relecture). Il répond 200 si tout fonctionne, 503 sinon, sans divulguer la cause, qui est journalisée.

12.4 Sauvegarde et restauration

Les scripts `scripts/backup.ps1` et `scripts/restore.ps1` s'appuient sur `pg_dump` et `pg_restore`, pour la base locale comme pour celle du conteneur. Chaque archive est relue avant d'être conservée, et seules les sept plus récentes sont gardées. La restauration s'exécute en une seule transaction : une archive corrompue laisse la base inchangée. Une base non vide n'est écrasée qu'avec l'option `-Force`. La procédure de vérification restaure la dernière sauvegarde dans une base neuve, puis contrôle le schéma avec `migrate -check`.

13 Conclusion

13.1 Bilan

Le back-end de TravelHub satisfait l'ensemble de l'énoncé. Le modèle relationnel combine des relations 1-N et une relation N-N porteuse d'attributs. Ses invariants sont garantis par PostgreSQL. Les mêmes fonctions sont offertes par une API REST documentée par OpenAPI et par une API GraphQL. Toutes deux partagent la logique métier, les permissions et la validation. Les deux services externes sont consommés avec délais, nouvel essai, cache et dégradation partielle. La sécurité suit l'OWASP API Security Top 10, et 724 tests automatisés couvrent 98 % du code.
Trois choix ont structuré le travail. Le filtrage systématique par `visible_to()` règle le contrôle d'accès au niveau objet par construction, plutôt que vue par vue. La double validation, applicative puis en base, rend les erreurs explicites sans sacrifier la cohérence en cas d'accès concurrent. Enfin, des services métier partagés ont réduit le coût de la seconde API à la seule couche de présentation.

13.2 Limites

— Aucun moteur de cache n'est configuré en production : Django utilise alors un cache en mémoire propre à chaque processus. Les réponses externes ne sont pas partagées entre les processus Gunicorn, et les compteurs de limites de requêtes non plus. Avec deux processus, la limite effective double.
— Les prévisions se limitent aux 16 prochains jours, et Countries fournit les noms de pays en anglais seulement.
— Les risques résiduels de la section 10 demeurent : énumération des courriels, largeur des requêtes GraphQL non bornée, documentation OpenAPI publique.
— La pagination GraphQL par décalage se dégrade sur de très grandes listes ; une pagination par curseur serait plus robuste.

13.3 Pistes pour le TP2

Le front-end pourra s'appuyer directement sur l'existant. GraphQL alimentera les écrans riches, comme la vue d'un voyage avec météo et pays, en une requête. Il suffira d'ajouter son origine à `CORS_ALLOWED_ORIGINS`. Les codes d'erreur stables et le champ `details` permettront d'afficher les messages au bon champ de formulaire, et l'en-tête `X-Request-ID` de relier une erreur affichée à son journal. Côté serveur, un cache partagé comme Redis lèverait la principale limite relevée ci-dessus.

Références

[1] Trevor Blades. Countries : a public GraphQL API for information about countries, continents, and languages. `https://github.com/trevorblades/countries`, 2026. Point d'accès : `https://countries.trevorblades.com/graphql`. Consulté le 23 octobre 2026. pages 17

[2] Tom Christie and Encode OSS. Django REST framework. `https://www.django-rest-framework.org/`, 2026. Version 3.18. Consulté le 23 octobre 2026. pages 6, 20

[3] Django Software Foundation. Deployment checklist. `https://docs.djangoproject.com/en/5.2/howto/deployment/checklist/`, 2025. Consulté le 23 octobre 2026. pages 32

[4] Django Software Foundation. Django 5.2 documentation. `https://docs.djangoproject.com/en/5.2/`, 2025. Consulté le 23 octobre 2026. pages 6

[5] drf-spectacular contributors. drf-spectacular : Sane and flexible OpenAPI 3 schema generation for Django REST framework. `https://drf-spectacular.readthedocs.io/`, 2026. Version 0.30. Consulté le 23 octobre 2026. pages 6, 23

[6] Roy Thomas Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000. pages 27

[7] Graphene contributors. Graphene-django documentation. `https://docs.graphene-python.org/projects/django/en/latest/`, 2025. Version 3.2. Consulté le 23 octobre 2026. pages 6, 24

[8] GraphQL Foundation. GraphQL specification, October 2021 edition. `https://spec.graphql.org/October2021/`, 2021. Consulté le 23 octobre 2026. pages 24

[9] Jazzband. Simple JWT documentation. `https://django-rest-framework-simplejwt.readthedocs.io/`, 2025. Version 5.5. Consulté le 23 octobre 2026. pages 6, 31

[10] Michael B. Jones, John Bradley, and Nat Sakimura. JSON web token (JWT). RFC 7519, Internet Engineering Task Force, 2015. `https://www.rfc-editor.org/rfc/rfc7519`. pages 31

[11] Open-Meteo. Geocoding API. `https://open-meteo.com/en/docs/geocoding-api`, 2026. Consulté le 23 octobre 2026. pages 17

[12] Open-Meteo. Weather forecast API. `https://open-meteo.com/en/docs`, 2026. Consulté le 23 octobre 2026. pages 17

[13] OWASP Foundation. OWASP API security top 10 – 2023. https://owasp.org/API-Security/editions/2023/en/0x11-t10/, 2023. Consulté le 23 octobre 2026. pages 5, 32

[14] The PostgreSQL Global Development Group. PostgreSQL 18 documentation. https://www.postgresql.org/docs/18/, 2025. Consulté le 23 octobre 2026. pages 6